

ROS2

DAY 08 – CLI Tools

Essential Command-Line Tools (Beginner → Professional)

Objective

By the end of this manual, you will be able to:

- Inspect a running ROS 2 system without opening code
- Discover nodes, topics, services, actions, and parameters
- Debug why something is not working
- Understand *what is running*, *who talks to whom*, and *how*

This manual is tool-focused, not coding-focused.

Pre-Requisites

- How to build and source a ROS 2 workspace
 - How to run nodes using `ros2 run` or `ros2 launch`
 - Basic ROS 2 concepts (node, topic, service)
-

Step 1 – The ROS 2 CLI Mental Model

Think of ROS 2 CLI as:

A live inspector of the ROS graph

It allows you to:

- Observe a running system
- Query communication
- Test functionality
- Debug issues without modifying code

Every serious ROS debugging session starts with CLI tools.

Step 2 – Core `ros2` Command Structure

All ROS 2 CLI commands follow this pattern:

```
ros2 <entity> <verb> [options]
```

Examples:

- `ros2 node list`
 - `ros2 topic echo`
 - `ros2 service call`
-

Step 3 – Node Inspection Tools

3.1 List All Running Nodes

```
ros2 node list
```

Use this to answer:

- Is my node running?
 - Did launch actually start it?
-

3.2 Inspect a Node in Detail

```
ros2 node info /node_name
```

Shows:

- Published topics
- Subscribed topics
- Services
- Parameters

This command explains everything a node does.

Step 4 – Topic Inspection Tools

4.1 List Topics

```
ros2 topic list
```

Add message types:

```
ros2 topic list -t
```

4.2 Echo Topic Data

```
ros2 topic echo /topic_name
```

Use when:

- You want to see live messages
 - You want to verify data flow
-

4.3 Topic Information

```
ros2 topic info /topic_name
```

Shows:

- Publishers
 - Subscribers
 - Message type
-

4.4 Measure Topic Rate

```
ros2 topic hz /topic_name
```

Confirms:

- Whether messages are publishing
 - Frequency correctness
-

Step 5 – Interface Inspection Tools

5.1 List Message Types

```
ros2 interface list
```

5.2 Inspect a Message Definition

```
ros2 interface show std_msgs/msg/String
```

Critical for:

- Understanding message structure
 - Debugging incorrect field usage
-

Step 6 – Service Inspection & Testing Tools

6.1 List Services

```
ros2 service list
```

6.2 Show Service Type

```
ros2 service type /service_name
```

6.3 Inspect Service Definition

```
ros2 interface show package_name/srv/ServiceName
```

6.4 Call a Service Manually

```
ros2 service call /service_name package_name/srv/ServiceName "{field: value}"
```

This allows full testing without writing a client node.

Step 7 – Parameter Tools

7.1 List Parameters

```
ros2 param list
```

7.2 Get Parameter Value

```
ros2 param get /node_name parameter_name
```

7.3 Set Parameter at Runtime

```
ros2 param set /node_name parameter_name value
```

Used heavily for:

- Tuning
 - Experimentation
 - Runtime configuration
-

Step 8 – Action Inspection Tools

8.1 List Actions

```
ros2 action list
```

8.2 Show Action Type

```
ros2 action info /action_name
```

8.3 Send an Action Goal

```
ros2 action send_goal /action_name package_name/action/ActionName "{goal_field: value}"
```

Allows testing without writing a client.

Step 9 – Launch & System-Level Tools

9.1 List Packages

```
ros2 pkg list
```

9.2 Find Package Location

```
ros2 pkg prefix package_name
```

9.3 Launch Files

```
ros2 launch package_name launch_file.py
```

Step 10 – Debugging Workflow (Professional Habit)

When something fails:

1. `ros2 node list` → Is it running?
2. `ros2 topic list` → Is data flowing?
3. `ros2 topic echo` → Is data correct?
4. `ros2 node info` → Who connects to whom?
5. `ros2 param get` → Are parameters correct?
6. `ros2 service call` → Can I trigger logic?

This workflow solves 90% of ROS issues.
